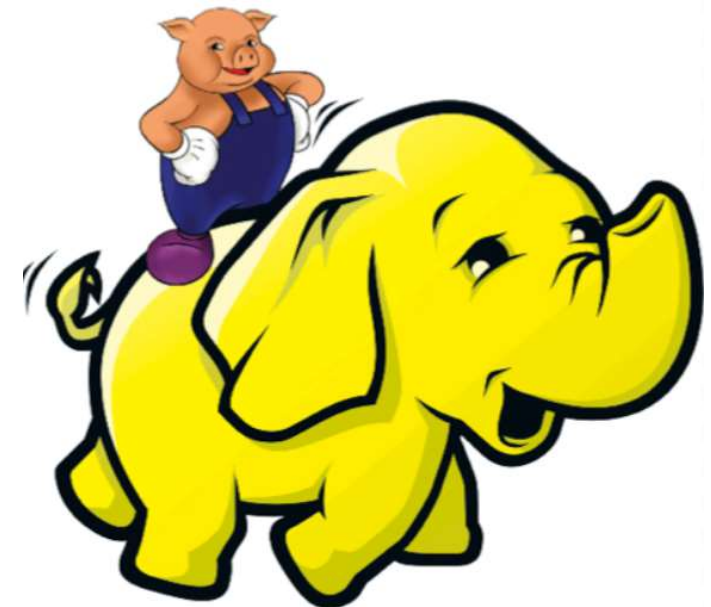
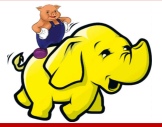


CH 4 PIG

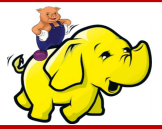
- ☐ Presentation
- ☐ Pig vs MapReduce
- ☐ Architecture & interaction
- ☐ Data Type & schema
- ☐ Pig Latin
- ☐ Exercises





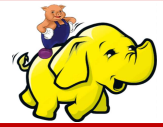
What is "Pig"?

- Pig is a **high-level scripting** language that uses Apache Hadoop as its software platform.
- Pig allows users to write programs that manipulate complex data without necessarily knowing Java.
- Pig is a language developed by the Yahoo team to overcome the complexity of programming in Java using the MapReduce paradigm.
- Pig is designed to quickly program data manipulation programs.
 - 01 Pig line = 20 lines (Java)
- development time (Pig) = 1/16 development time (Java)
- Pig is an imperative data manipulation language (as opposed to SQL)

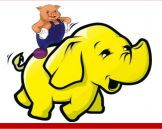


- Pig **works with data from many sources**, including structured and unstructured data, and stores the results in HDFS: Hadoop Data File System.
- Its name? Pig" is inspired by the domestic animal "pig", which **swallows** all types of food. By analogy, Apach Hadoop **Pig processes all types of data** (structured, semi-structured, unstructured).
- **Pig scripts are translated into a series of Mapreduce tasks** that are executed on the Apache Hadoop cluster.
- Pig is different from procedural and/or object-oriented programming languages. **It is a data flow processing language**
- Pig is a query and data processing language.
- Pig is similar to SQL in that it provides built-in functions for processing data flows (joins, concatenation, etc.).
- Pig has its own data structures (atom, tuples, bag, map).

Pig Vs MapReduce

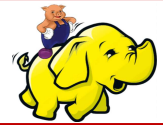


Pig	MapReduce
Apache Pig is a data flow language	Mapreduce is a data processing paradigm
It's a high-level language.	Mapreduce is low-level and rigid.
Performing a "Join" operation with Apache Pig is fairly straightforward.	In Mapreduce, it is quite difficult to perform a junction operation between data.
Any novice programmer with a basic knowledge of SQL can work easily with Apache Pig	Advanced knowledge of Java is required to work with Mapreduce.
Apache Pig uses a multi-request approach, which considerably reduces code length	Mapreduce will require almost 20 times as many lines to perform the same task.

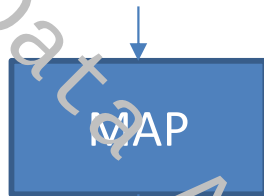


- Exercise: counting a child's objects
 - a child plays with his Hadoop cuddly toy
 - a child playing with a red ball
 - a child is drawing on the walls with his pen
 - a child is riding his bicycle

PIG Vs MapReduce

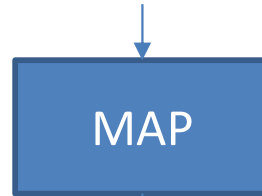


un enfant joue avec
son peluche Hadoop



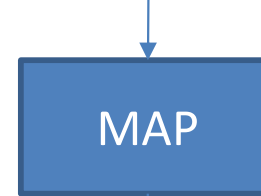
Un : 1
Enfant : 1
Joue : 1
Avec :
son : 1
Peluche : 1
Hadoop : 1

un enfant joue avec
son ballon rouge



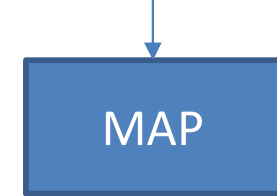
un : 1
Enfant : 1
Joue : 1
Avec : 1
Son : 1
Ballon : 1
Rouge : 1

un enfant dessine avec
son stylo sur les murs

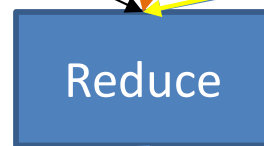


un : 1
Enfant : 1
Dessine : 1
Avec : 1
son : 1
Stylo : 1
Sur : 1
les : 1
Murs : 1

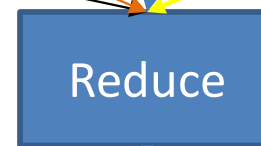
un enfant monte sur
sa bicyclette



Un : 1
Enfant : 1
Monte : 1
sur : 1
Sa : 1
Bicyclette : 1

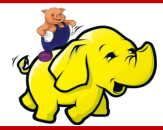


Un : 4
Enfant : 4
Joue : 2
Avec : 3
son : 3
Hadoop : 1



ballon : 1
Peluche : 1
Stylo : 1
Bicyclette : 1
Rouge : 1
...

PIG Vs MapReduce



-- Load : télécharger le fichier 'enfant' et nommer chaque champ 'line'
input = load 'enfant' as (line);

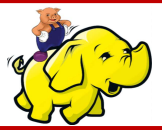
-- TOKENIZE : segmenter la ligne en champs isolés
-- flatten : créer un enregistrement 'mots' des champs isolés générés par TOKENIZE. Les champs isolés sont appelés 'mot'
mots = foreach input generate flatten(TOKENIZE(line)) as (mot);

-- group : grouper les mots
grp = group mots by mot;

-- Count : comptage
cntd = foreach grp generate group, COUNT(mots);

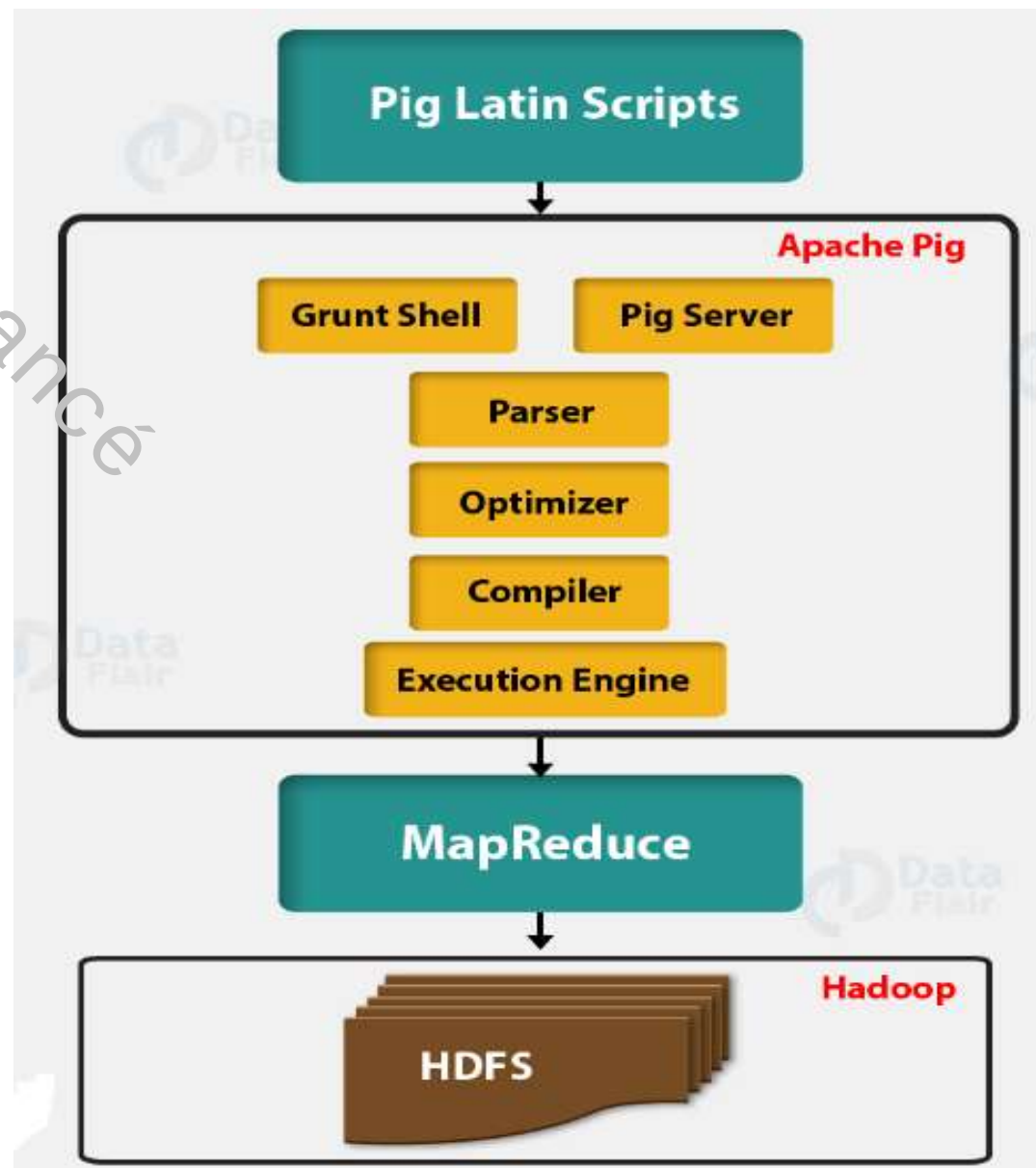
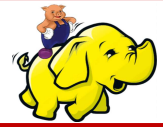
-- dump : afficher le résultat.
dump cntd;

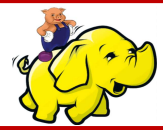
➡ Only 4 lines of Pig replace more than 80 lines of Mapreduce (java)



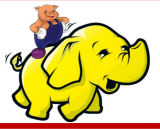
- Pig Latin is a language similar to SQL. There are several keywords in common (JOIN , ORDER , LIMIT . . .) but their principles are different:
 - In **SQL**, we construct queries **that describe the data to be obtained**. We don't know how the SQL engine will calculate the result. All we know is that, internally, the query will be broken down into loops and comparisons on the data, making the best possible use of the indexes.
 - In **Pig Latin**, you build programs that contain instructions. They **describe exactly how the result is to be obtained**, what calculations are to be made and in what order.
 - **Pig was designed** for the **uncertain data of Hadoop**, while SQL runs on perfectly sound DBMSs.

Pig : architecture





- Pig is essentially made up of **two modules**
 - **Pig Latin**: high-level procedural language
 - **Pig Engine**: automatically parse, optimise, compile and execute PigLatin scripts as a series of MapReduce jobs within a Hadoop cluster.
- **Parser (verification)**
 - "Parser essentially checks the syntax of the script, does type checking, and other miscellaneous checks.
 - The output of Parser is a **DAG (directed acyclic graph)** that represents Pig Latin statements and logical operators.
 - The logical operators in the script are represented by nodes and the data flows are represented by edges in the DAG (the logical plane).
- **Optimiser**
 - Next, the logic map (DAG) is passed to the logic optimizer. It performs logical optimizations



- **Compiler**

The compiler then compiles the optimized logic plan into a series of MapReduce jobs.

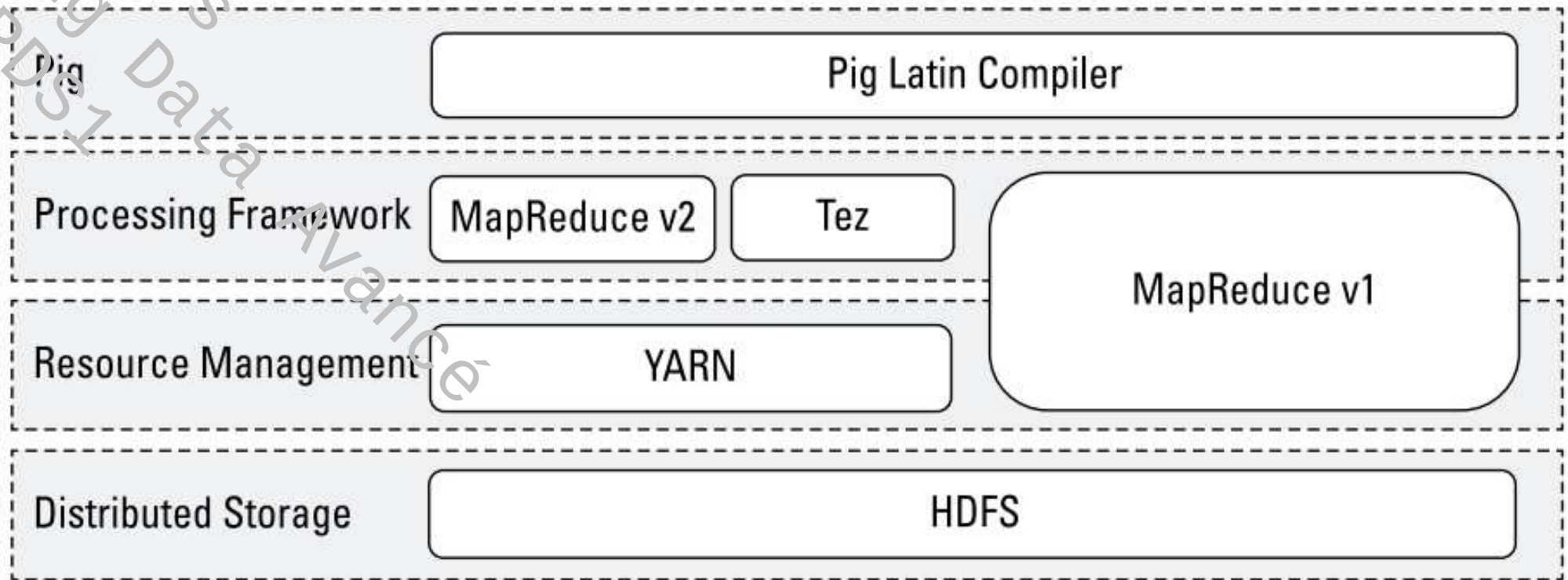
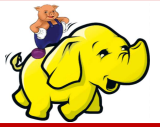
- **Execution engine**

Finally, all the MapReduce jobs are submitted to Hadoop in a sorted order.

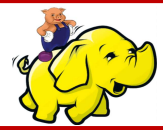
- **Note:**

Pig is installed on the user's **local machine**. It does not need to be installed on the Hadoop cluster.

PIG : architecture



- Pig programs can run on Mapreduce v1 or Mapreduce v2 without any changes to the code.
- Pig scripts can run using the Tez API. Apache Tez provides a more powerful execution framework than Mapreduce.



■ Apache Pig execution modes

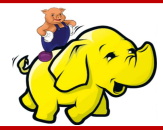
Apache Pig can be run in two modes: **local mode** and **HDFS mode**.

- Local mode

In this mode, all files are installed and run from the local host and the local file system. There is no need for Hadoop or HDFS. This mode is generally used for testing purposes.

- Mapreduce Mode

Mapreduce is the mode where we load or process data that exists in the Hadoop file system (HDFS). In this mode, each time Pig Latin instructions are executed to process the data, a Mapreduce task is invoked in the back-end to perform a particular operation on the data that exists in the HDFS.



- **Interactive mode (Grunt shell)**

This involves interacting with the Pig shell. In this mode, you can run MapReduce processes or interact with HDFS.

execution :

\$ **pig -x local** or \$ **pig -x mapreduce**
grunt>

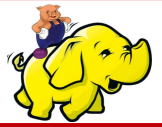
- **Batch Mode (Script)**

You can run Apache Pig in batch mode by writing the Pig Latin script in a single file with the **.pig** extension

\$**pig -x local script.pig** or \$**pig -x mapreduce script.pig**

- **Integrated mode (UDF)**

Apache Pig provides the ability to define functions (User Defined Functions) in other programming languages such as Java and use them as APIs.



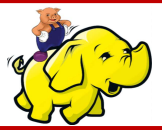
- ▶ Pig's data types can be divided into two categories

- **Atom**: containing a single value
- **Complex**: containing other types.

■ Atom

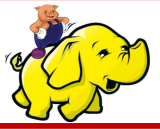
Atom represents the simple data types found in almost all programming languages.

- Int: represents integers. encoded on 4 signed bytes = 32bits.
- Long: represents long integers. encoded on 8 bytes = 64 signed bits.
- Float: represents real data. It is coded on 4 bytes = 32 bits.
- Double: designates real data (double precision). It is coded on 8 bytes = 64 bits.
- Chararray: used to represent strings or an array of characters.
- Bytearray, Boolean, BigInteger, BigDecimal



■ Tuple

- It is an ordered collection.
 - A tuple is made up of several fields, each of which can be of any type.
 - The elements of a tuple are not necessarily of the same type.
 - A field can be accessed through its position.
 - Pig allows to associate a **schema with a tuple** to *describe the type* of each field and *give it a name*. In this case, it is possible *to access a field by its name*.
-
- Example
 - ('MPDS1', 10, 4)
 - (2, 5, 6)



■ **Bag**

- An unordered collection of tuples (no notion of position)

- Example

`{('MPDS', 1), ('MR', 'Info', 2), ('DepInfo', 'FSB')}`

■ **Map**

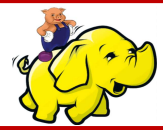
- It is a dictionary made up of a key and a value.

- The key is of **chararray** type, and is used as an index to find the associated value.

- The value is considered to be a **bytearray**.

- Exemple

`[ 'nom'# 'Kamel', 'age'# 25]`



- Extraction of fields of a complex type: a projection operator must be used depending on the type of data

- Tuple → operator : **(.)** the dot

```
A = load 'input' as (t:tuple(x:int, y:int));  
B = foreach A generate t.x, t.$1;
```

- Map → operator : **(#)**

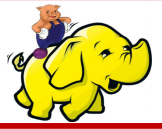
```
bball = load 'baseball' as (name:chararray, team:chararray,  
position:bag{t:(p:chararray)}, bat:map[]);  
avg = foreach bball generate bat#'batting_average';
```

- Bag → operator : **(.)** BUT

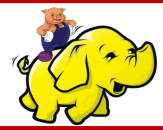
on crée un Bag et non pas un scalaire

```
A = load 'input' as (b:bag{t:(x:int, y:int)});  
B = foreach A generate b.x;
```

-- B est un Bag contenant uniquement les valeurs (x)



- Pig includes the concept of a **Null** data element.
- Data of any type can be Null.
- The concept of Null in Pig is the same as in SQL, which is completely different from the concept of Null in C, Java, Python, etc.
- In Pig, a Null data element means that the value **is unknown**. This may be because the data is missing, an error has occurred in processing, etc.

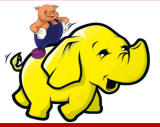


- **Schemas are used to associate field names and types.** They are optional but recommended, as they enable error control and better optimisation of code execution.
- The simplest way of communicating the data schema to Pig is to explicitly tell Pig the data type when it is loaded.
- Example

```
students = load 'MPDS1' as (lastname:chararray,  
    firstname:chararray, age:int);
```

Pig expects the data to have 3 fields. **If there are more**, it will **truncate** the extra fields. **If there are fewer**, it will augment with **Nulls**.
- It is also possible to specify the schema without giving explicit data types. In this case, the data type is assumed to be **bytearray**

```
etudiants = load 'MPDS1' as (nom, prénom, age);
```



- A relation is a Tuple bag

example

```
etudiants = load 'MPDS1' as (nom:chararray, prénom:chararray, age:int);
```

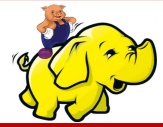
- Pig references a field by its position or name.
 - **access by position**: use of the \$ operator followed by the position number. position n is marked by $$(n-1)$.
 - **access by name**: the name is the field identifier.

- Exemple

```
X = FOREACH etudiants GENERATE prénom, $1;  
DUMP X
```

Access with key

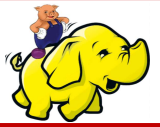
Access with position



- Pig Latin is a **data flow language**. Each processing step results in a **new set of data, or relationship**.
- Example
 - A = load 'data'
 - A: the name of the relationship that results from loading the data from the dataset.
 - A : relation name is called an alias.
 - Relation names look like variables, but they are not.
- Example

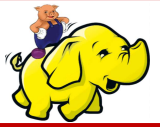
It looks here as if we are reassigning A, but really we are creating new relationships called A, and losing track of the old relationships called A.

```
A = load 'articles' AS (catégorie, type, date, prix);  
A = filter A by prix > 100;  
A = foreach A generate UPPER(type);
```



- Note: Case sensitivity
 - Pig Latin is not case-sensitive: **LOAD** and **load** are identical
 - Variables are case-sensitive: **TYPE** is different from **type**
- Loading data: **load**
 - data is loaded using the load instruction
 - by default, Pig loads data from HDFS using a predefined path.
 - it is also possible to load data via another storage system such as HBase.

```
A = load 'MPDS1' using HBaseStorage(); --Hbase
A = load 'MPDS1' using PigStorage(','); --HDFS (par défaut), délimiteur (,)
A = load 'MPDS1' as (nom:chararray, prénom:chararray, age:int);
/*
* Puisque la méthode n'est pas déclarée → PigStorage →HDFS
* As : pour spécifier le schéma
*/
```



■ Storage: **store**

- By default, Pig stores data on HDFS in a file using PigStorage.
- It is possible to specify other functions such as HBaseStorage.
- Writing processing results requires the relative or absolute path of the file.

```
store res into '/data/resultatTraitement'; -- chemin relatif
```

```
store res into hdfs://data.process.com/data/resultatTraitement; --chemin absolu
```

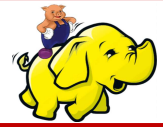
```
store res into '/data/resultatTraitement' using PigStorage(); --avec précision HDFS
```

■ Display: **dump**

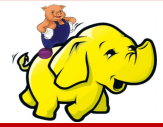
- Display data on screen

```
dump res
```


Pig Latin: arithmetic operations



Operation	Symbol	
addition	+	
subtraction	-	
multiplication	*	
division	/	
modulo	%	Le reste de division (int, long)
bincond	? :	Condition booléenne Doit être entre ().
case	CASE WHEN THEN ELSE END	Exécution conditionnée



Example : A

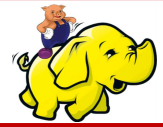
```
A = LOAD 'data' AS (f1:int, f2:int);
DUMP A;
(10,1)
(10,3)
(10,6)

X = FOREACH A GENERATE f2, (
    CASE f2 % 2
        WHEN 0 THEN 'even'
        WHEN 1 THEN 'odd'
    END
);

DUMP X;
(1,odd)
(3,odd)
(6,even)
```

For more information see the official site: Pig

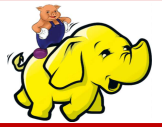
<http://pig.apache.org/docs/latest/basic.html#arithmetic-ops>



- Pig offers a number of data transformation tools: **sorting, grouping, joining, filtering, etc.**
- **Foreach**
 - Applies a set of operations to each record in the data pipeline (hence the name foreach)

This code downloads the data, but deletes all the fields except the user and the record identifier

```
A = load 'input' as (user:chararray, id:long, address:chararray,  
phone:chararray, preferences:map[]);  
B = foreach A generate user, id;
```

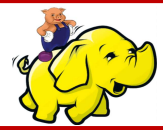


Filter

- Selecting a tuple from a relation according to a condition
- Syntax: `alias = FILTER alias BY expression;`

```
A = LOAD 'data' AS (f1:int, f2:int);  
DUMP A;  
(10,1)  
(10,3)  
(10,6)
```

```
B = filter A by f2 > 2  
Dump B  
(10,3)  
(10,6)
```



group

- Group the data

Détails des étudiants

Kamel, 22, bizerte, 10,61

Olfa, 23, tunis, 11,96

Karim, 21, kairouan, 13,02

Jihene, 21, sousse, 11,05

```
A = LOAD 'hdfs://localhost:9000/pig_data/liste_etudiant.txt' USING  
PigStorage(',') as (nom:chararray, age:int, ville:chararray, moyenne:float);
```

```
group_A1 = GROUP A by age;
```

```
Dump group_A1
```

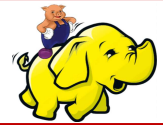
```
(21, {(Karim, 21, kairouan, 13,02), (Jihene, 21, sousse, 11,05)})
```

```
(22, {(Kamel, 22, bizerte, 10,61)})
```

```
(23, {(Olfa, 23, tunis, 11,96)})
```

Voir aussi

```
group_A2 = GROUP A by (age, ville);
```



■ join

- Join the data
- Syntax : Relation3 = JOIN Relation1 BY key, Relation2 BY key ;

D1: Détails des étudiants

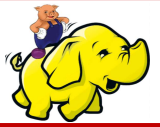
Kamel, 22, bizerte, 10.61
Olfa, 23, tunis, 11.96
Anouar, 29, Bizerte, 13.02
Jihene, 21, sousse, 11.05

D2: Détails des étudiants

Kamel, football
Olfa, natation
Anouar, **spéléologie**
Jihene, handball

```
D1 = LOAD 'hdfs://localhost:9000/pig_data/D1.txt' USING PigStorage(',') as  
(nom:chararray, age:int, ville:chararray, moyenne:float);  
D2 = LOAD 'hdfs://localhost:9000/pig_data/D2.txt' USING PigStorage(',') as  
(nom:chararray, sport:chararray);
```

```
D1D2 = join D1 by nom, D2 by nom ;  
Dump D1D2  
(Kamel, 22, bizerte, 10.61,football)  
(Olfa, 23, tunis, 11.96, natation)  
(Anouar, 29, Bizerte, 13.02, spéléologie)  
(Jihene, 21, sousse, 11.05, handball)
```



■ union

- data concatenation
- Syntaxe : Relation3 = union Relation1, Relation2 ;

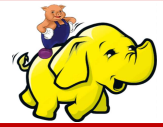
D1: Détails des étudiants

Kamel, 22, bizerte, 10.61
Olfa, 23, tunis, 11.96
Karim, 21, kairouan, 13.02
Jihene, 21, sousse, 11.05

D2: Détails des étudiants

Hanene, 24, bizerte, 10.32
Moez, 21, tunis, 11.15

```
D1D2 = union D1, D2;  
Dump D1D2  
(Kamel, 22, bizerte, 10.61)  
(Olfa, 23, tunis, 11.96)  
(Karim, 21, kairouan, 13.02)  
(Jihene, 21, sousse, 11.05)  
(Hanene, 24, bizerte, 10.32)  
(Moez, 21, tunis, 11.15)
```



■ limit

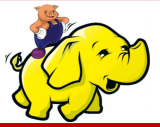
- Limit the number of output tuples
- Syntax : alias = LIMIT alias n;

```
D1D2 = union D1, D2;
D1D2_limit = limit D1D2 3
Dump D1D2_limit
(Kamel, 22, bizerte, 10.61, football)
(Olfa, 23, tunis, 11.96, natation)
(Karim, 21, kairouan, 13.02, spéléologie)
```

■ split

- Split a relationship into two or more relationships
- Syntaxe : SPLIT alias INTO alias IF expression, alias IF expression;

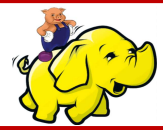
```
D1D2 = union D1, D2;
Split D1D2 into X if age >= 23, Y if moyenne >= 11.50
Dump X
(Olfa, 23, tunis, 11.96, natation)
(Hanene, 24, bizerte, 10.32)
Dump Y
(Olfa, 23, tunis, 11.96, natation)
(Karim, 21, kairouan, 13.02, spéléologie)
```

For further information, visit the official website of
Apache Pig

<http://pig.apache.org/docs/latest/basic.html#join-inner>

<https://pig.apache.org/docs/latest/func.html#sum>



■ Exercise

You are given a file called 'Aziza_Store' with the following contents:

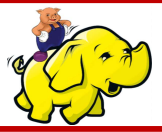
Bizerte, 1897, 5254, 251, 0.210, 8, 20

Jarzouna, 2541, 7863, 365, 0.655, 9, 22

Bizerte_Corniche, 563, 2988, 67, 0.150, 8, 21

```
A = load 'Magasin_Aziza_Bizerte.txt' as (ville, NBarticles,  
ChiffreAffaire, prixhigh, prixlow, ouverture, fermeture);  
diff1 = foreach A generate prixhigh - prixlow;  
diff2 = foreach A generate $3 - $4;
```

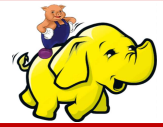
- 1) *Analyse the code and comment on the values of diff1 and diff2*
- 2) *Write a code in Pig Latin that sorts the shops according to their sales and then displays the store with the highest sales.*
- 3) *Display shops with more than 2000 items sold*
- 4) *Display the duration time of each shop has been open*



■ Exercise :

- Given a file named 'patients.txt' containing 4 columns separated by (:):
identifiant: name: age: disease
- You are asked to write a program in Pig Latin which displays the 10 patients whose age is between 15 and 25 in descending order.

Appendix: example of field referencing



```
cat data.log;

(jim, 6, 8,9) (Rach, 7,5,6)
(Dia, 1,4,7) (Livus, 3,7,5)
(Alex, 2,5,8) (Alain, 9,5,8)

A = LOAD 'data.log' AS (t1:tuple(name:chararray, t1a:int,
t1b:int,t1c:int),t2:tuple(name:chararray,
t2a:int,t2b:int,t2c:int));

DUMP A;

((jim, 6, 8,9), (Rach, 7,5,6))
((Dia, 1,4,7), (Livus, 3,7,5))
((Alex, 2,5,8), (Alain, 9,5,8))

X = FOREACH A GENERATE t1.name, t1.t1a,t2.$0;

DUMP X;

(Jim, 6,7)
(Dia, 1,3)
(Alex,2,9)
```

Appendix: Flow chart (organisation chart)

